Phase 4: System Analysis & Design

NutriVision

4.1. Problem Statement & Objectives:

Problem Statement

Modern dietary health applications often lack personalization, real-time synchronization, and AI-driven insights. Most existing solutions rely on static food databases and manual input, which limits their effectiveness for users seeking tailored nutritional advice. Additionally, many apps fail to provide dynamic recipe alternatives or integrate cross-device meal tracking seamlessly.

Objectives

NutriVision aims to solve these challenges by:

- Analyzing meal descriptions with AI to provide accurate nutritional breakdowns.
- Generating personalized recipe alternatives using prompt engineering tailored to dietary restrictions.
- Synchronizing meal logs in real time across multiple devices via Firebase Cloud Firestore.
- Delivering an intuitive, responsive UI built with Flutter (GridView, Stack, charts).
- Ensuring reliability and performance through structured testing (unit, widget, integration).
- Providing secure networking with Dio interceptors for API calls.

4.1.1. Use Case Descriptions:

Use Case 1: Log Meal

- Actor: User
- Precondition: User is logged into the app.
- Main Flow:
    1. User enters a meal description (text).
    2. System sends data to external nutritional API via Dio.
    3. API returns nutritional breakdown.
    4. System stores meal data in Firebase Firestore.
    5. Meal appears in user's history with timestamp.
- Postcondition: Meal is logged and visible in history.

Use Case 2: Get Recipe Alternative

- Actor: User
- Precondition: Meal is logged or user requests a recipe suggestion.
- Main Flow:
    1. User selects a meal/snack for alternative suggestion.
    2. System uses AI prompt engineering to generate healthier recipe options.
    3. Suggested recipe is displayed with nutritional comparison.
- Postcondition: User receives personalized recipe alternatives.

Use Case 3: Sync Meal Logs

- Actor: User, Firebase Cloud Firestore
- Precondition: User has logged meals on one device.
- Main Flow:
    1. System updates Firestore with new meal entry.
    2. Firestore syncs data across devices in real time.
    3. User opens app on another device and sees updated meal history.
- Postcondition: Meal logs are consistent across all devices.

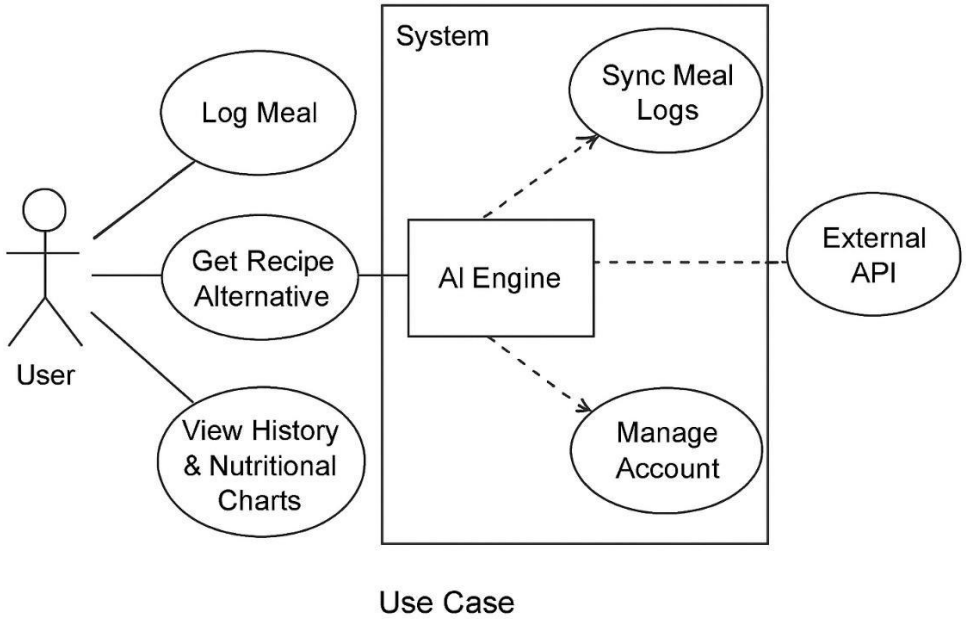
Use Case 4: View History & Nutritional Charts

- Actor: User
- Precondition: User has logged meals.
- Main Flow:
    1. User navigates to “Meal History.”
    2. System retrieves meal data from Firestore.
    3. UI displays meals in GridView with nutritional charts.
- Postcondition: User can view past meals and analyze nutrition trends.

Use Case 5: Manage Account

- Actor: User
- Precondition: User has registered an account.
- Main Flow:
    1. User updates dietary preferences (e.g., vegetarian, low-carb).
    2. System saves preferences in Firestore.
    3. AI suggestions adapt to updated preferences.
- Postcondition: User profile and dietary settings are updated.

Use case diagram:



4.1.2. Functional & Non-Functional Requirements:

Functional Requirements (System Capabilities)

These define what the system must do to support user and stakeholder needs:

Functionality	Description
User Account Management	Register, login, and manage dietary preferences.
Meal Logging	Input meal descriptions (text), edit, delete, and timestamp entries.
Nutritional Analysis	Send meal data to external API via Dio; receive and display breakdown.
AI Suggestions	Use prompt engineering to generate personalized recipe alternatives.
Meal History & Charts	Display logged meals and nutritional trends using GridView and charts.
Real-Time Sync	Sync meal logs across devices using Firebase Cloud Firestore.
Testing Framework	Support unit, widget, and integration testing for all modules.

Non-Functional Requirements (System Constraints)

Category	Requirement
Performance	Nutritional analysis response time < 2 seconds.
Security	Secure API calls with Dio interceptors; Firebase rules for data protection.
Usability	Intuitive UI with responsive design (Flutter GridView, Stack).
Reliability	≥ 99% uptime; stable cross-device synchronization.
Scalability	Support growing user base without performance degradation.
Maintainability	Modular architecture (MVC), clean codebase, documented components.
Accessibility	UI accommodates diverse users (contrast, font size, tap targets).

4.1.3. Software Architecture:

Architecture Style: MVC (Model–View–Controller)

NutriVision follows the MVC architecture, which separates concerns across three layers:

Layer	Responsibility
Model	Manages data: meal entries, nutritional values, user preferences.
View	Displays UI: meal history, charts, recipe suggestions.
Controller	Handles logic: API calls, AI prompts, Firebase sync, user actions.

System Components

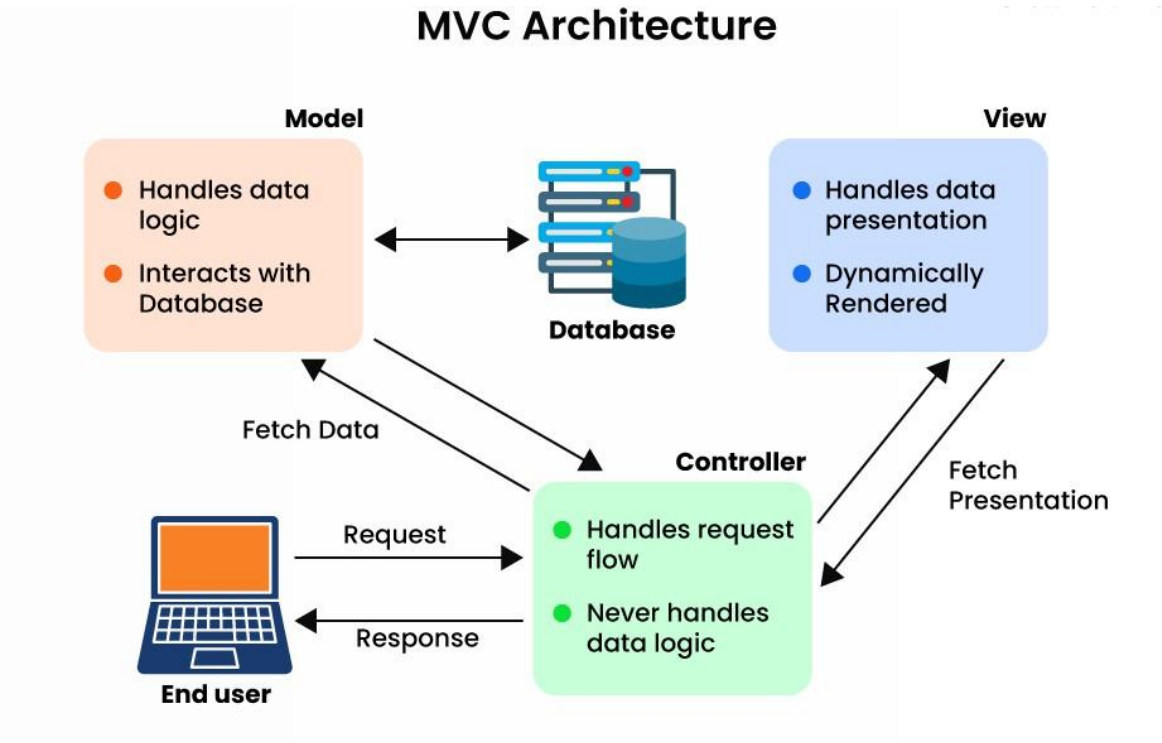
1. User Interface (View Layer)
    - Built with Flutter using GridView, Stack, and charts.
    - Displays meal logs, nutritional breakdowns, and recipe suggestions.
    - Responsive across devices.
2. Data Model (Model Layer)
    - Represents meals, user profiles, and nutritional data.
    - Stores structured objects for Firebase and API integration.
3. Controller Logic (Controller Layer)
    - Handles user input, API requests via Dio, and AI prompt generation.
    - Manages Firebase sync and error handling.
    - Coordinates between Model and View.

4. External Nutritional API
    - Provides nutritional analysis based on meal descriptions.
    - Integrated securely via Dio with interceptors.
5. AI Engine
    - Generates personalized recipe alternatives using prompt engineering.
    - Adapts suggestions based on user preferences.
6. Firebase Cloud Firestore
    - Stores meal logs and user data.
    - Enables real-time synchronization across devices.

Component Interactions

- User logs a meal → Controller sends to API → Model stores result → View displays analysis.
- User requests recipe alternative → Controller sends prompt to AI → Model receives suggestion → View displays recipe.
- User updates preferences → Controller updates Firebase → Model reflects changes → View adapts suggestions.

MVC diagram:


MVC Architecture
Model
Handles data logic
Interacts with Database
Database
View
Handles data presentation
Dynamically Rendered
Fetch Data
Controller
Fetch Presentation
Request
Handles request flow
Never handles data logic
Response
End user

4.2.1. Entity-Relationship Diagram (ERD):

Core Entities & Relationships

Entity	Key Attributes	Relationships
User	user_id (PK), name, email, password, dietary_preferences	One-to-Many with Meal, One-to-Many with RecipeSuggestion
Meal	meal_id (PK), user_id (FK), description, timestamp	Many-to-One with User, One-to-One with NutritionalData
NutritionalData	nutrition_id (PK), meal_id (FK), calories, protein, carbs, fats, vitamins	One-to-One with Meal
RecipeSuggestion	suggestion_id (PK), user_id (FK), original_meal, suggested_recipe, AI_prompt	Many-to-One with User

Relationships

- A User can log multiple Meals.
- Each Meal has one NutritionalData entry.
- A User can receive multiple RecipeSuggestions.
- Each RecipeSuggestion is linked to a user and references an original meal.

4.2.2. Logical & Physical Schema:

Logical Schema (Normalized Tables)

User

- user_id (PK)
- name
- email
- password
- dietary_preferences

Meal

- meal_id (PK)
- user_id (FK)
- description
- timestamp

NutritionalData

- nutrition_id (PK)
- meal_id (FK)
- calories
- protein
- carbs
- fats

- vitamins

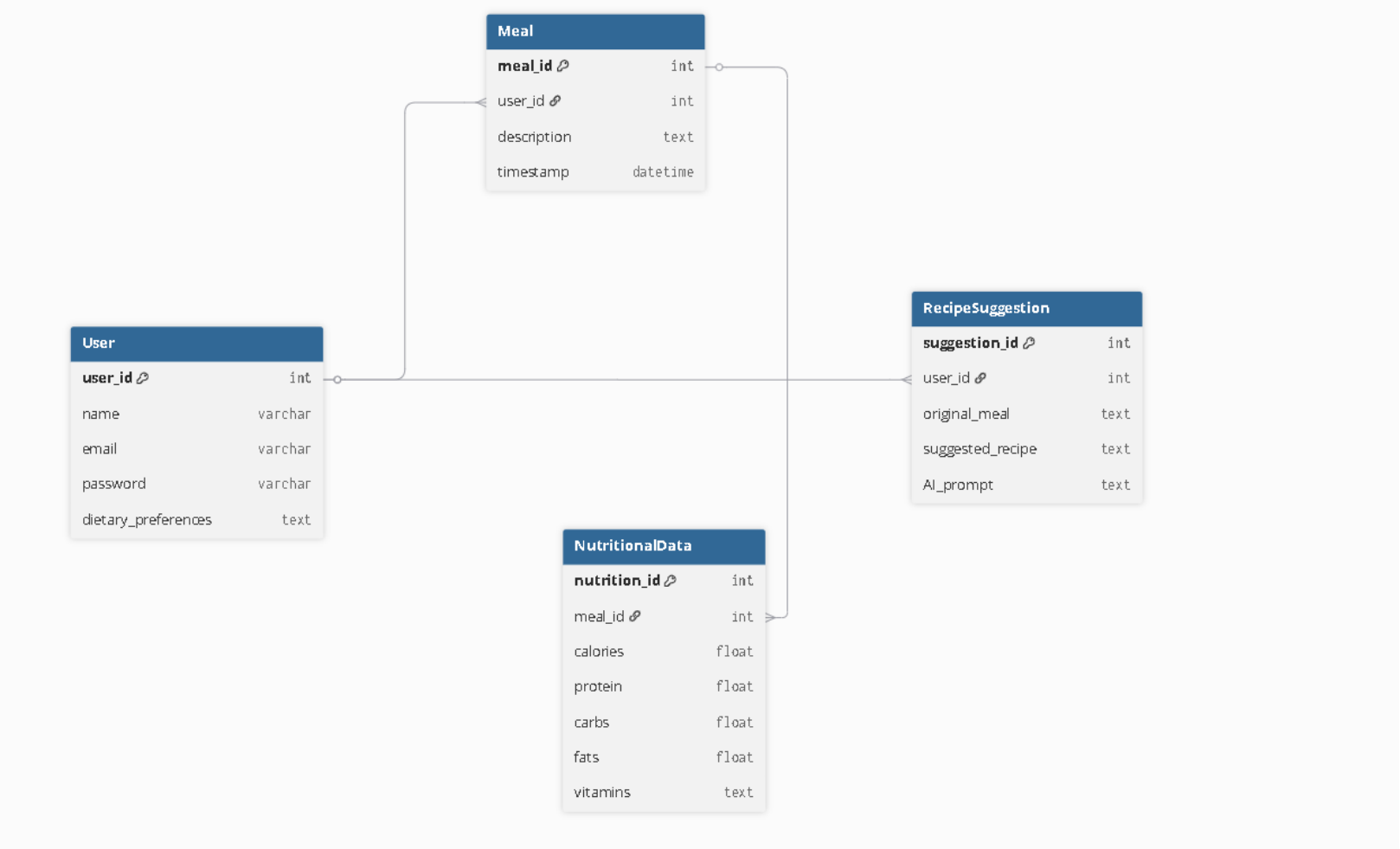
RecipeSuggestion

- suggestion_id (PK)
- user_id (FK)
- original_meal
- suggested_recipe
- AI_prompt

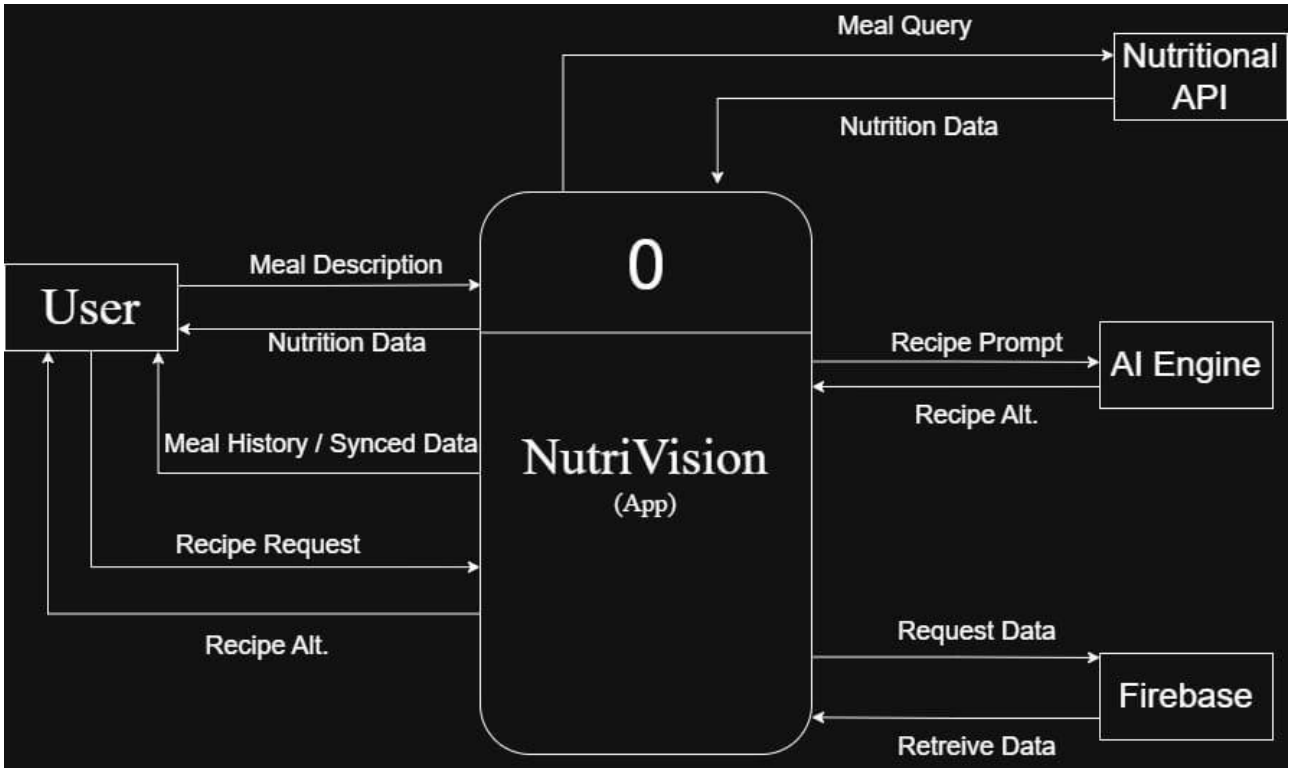
Physical Schema Considerations

- Normalization:
  - 1NF: All attributes are atomic.
  - 2NF: No partial dependencies (each non-key depends on full PK).
  - 3NF: No transitive dependencies (e.g., vitamins not dependent on calories).
- Indexes:
  - On user_id, meal_id, and timestamp for fast retrieval.
- Security:
  - Passwords hashed and salted.
  - Firebase rules enforce access control per user.

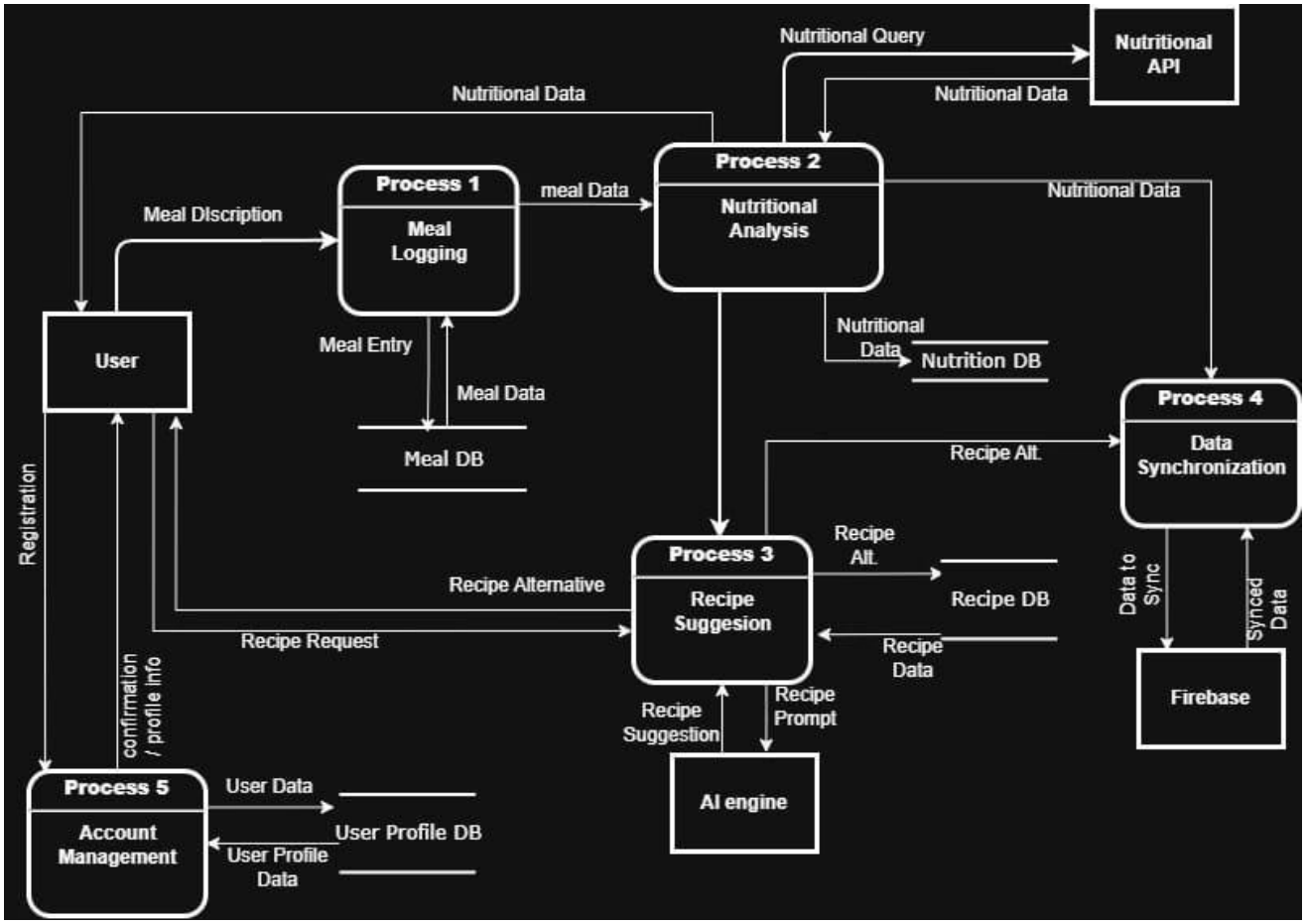
ER diagram:


Meal
meal_id int
user_id int
description text
timestamp datetime
User
user_id int
name varchar
email varchar
password varchar
dietary_preferences text
RecipeSuggestion
suggestion_id int
user_id int
original_meal text
suggested_recipe text
AI_prompt text
NutritionalData
nutrition_id int
meal_id int
calories float
protein float
carbs float
fats float
vitamins text

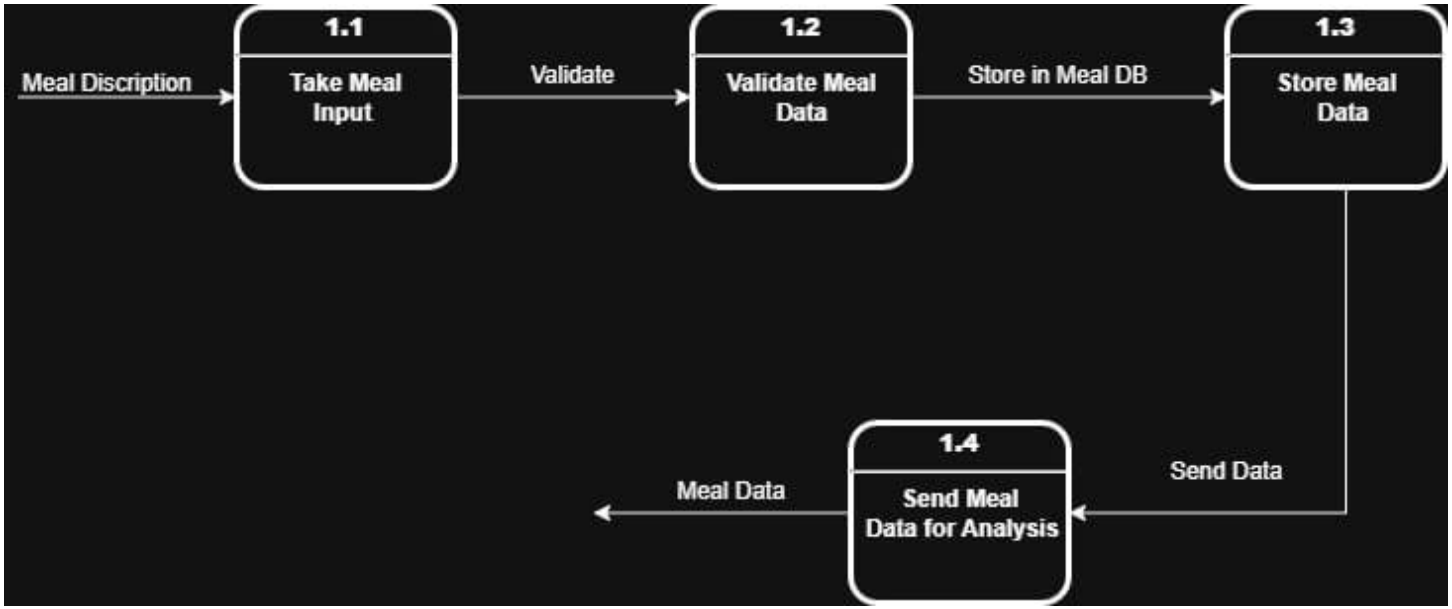
4.3.1. DFD diagram (Context level):



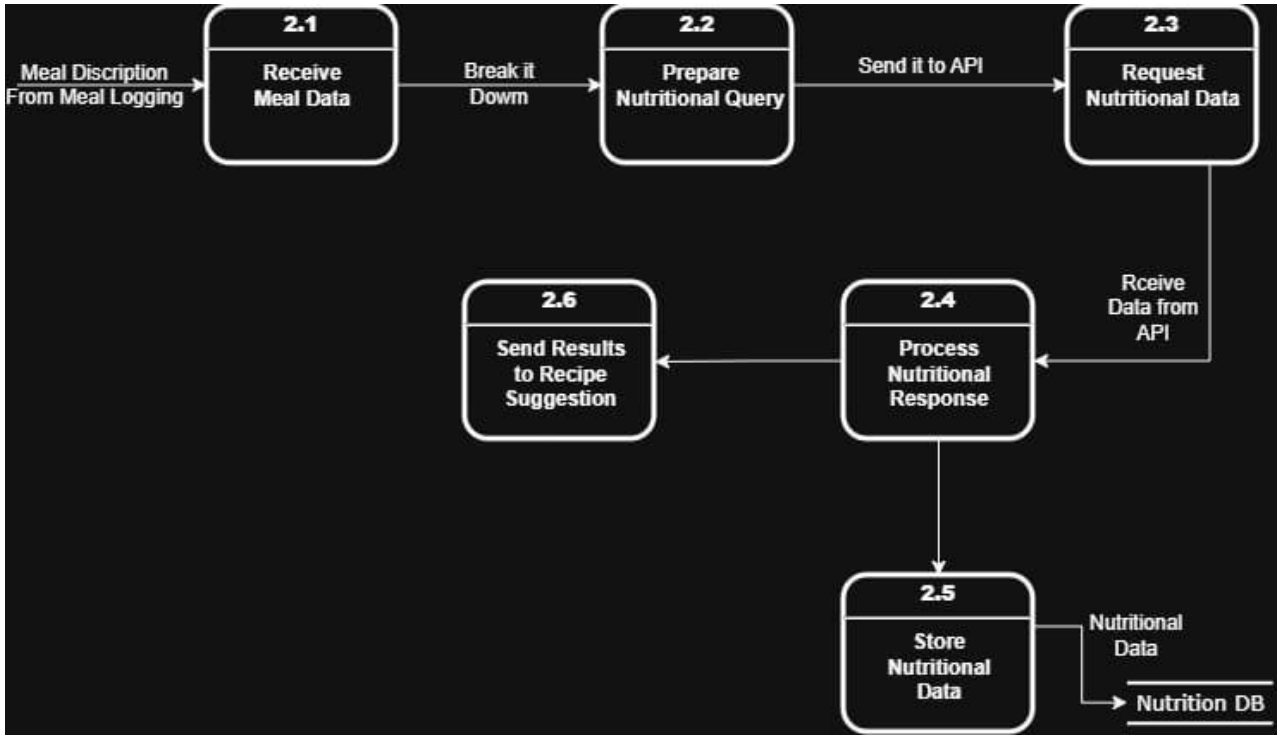
DFD diagram (Level 1):



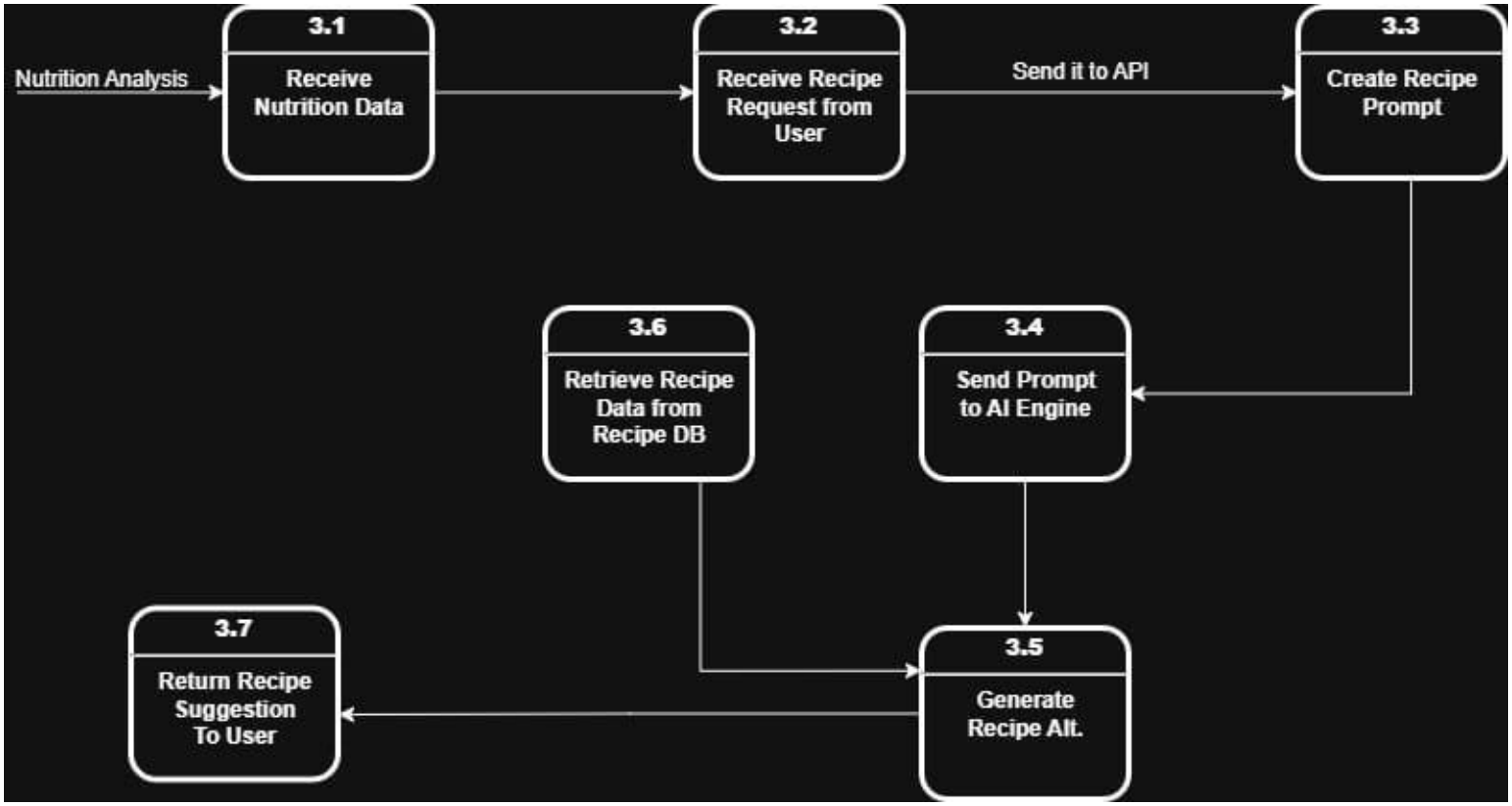
DFD diagram (Level 2, process 1):



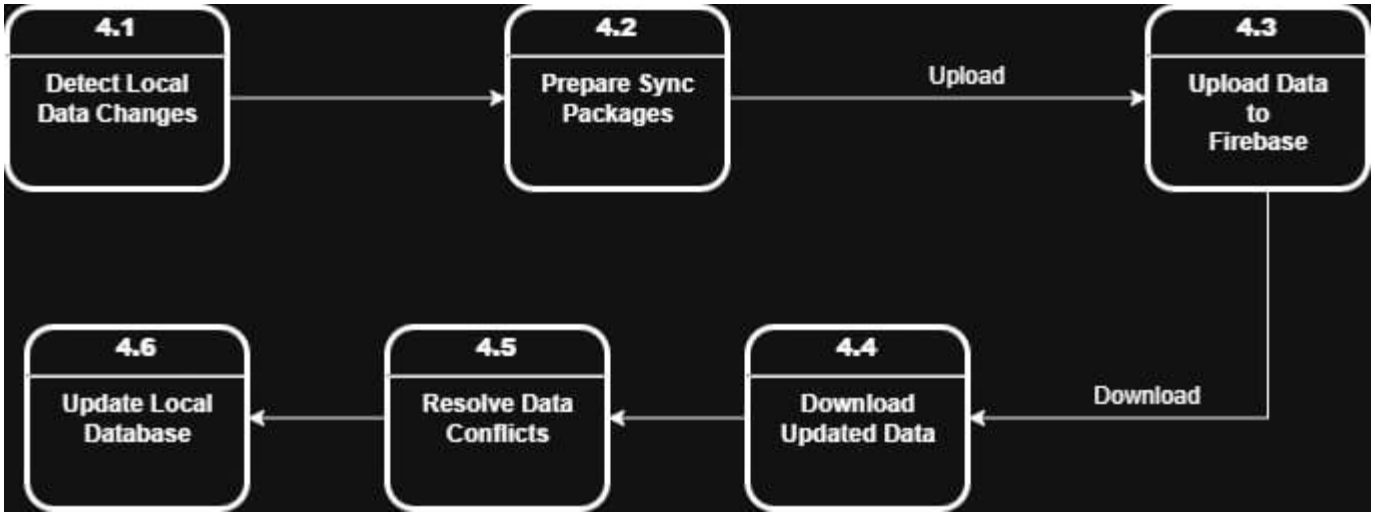
DFD diagram (Level 2, process 2):



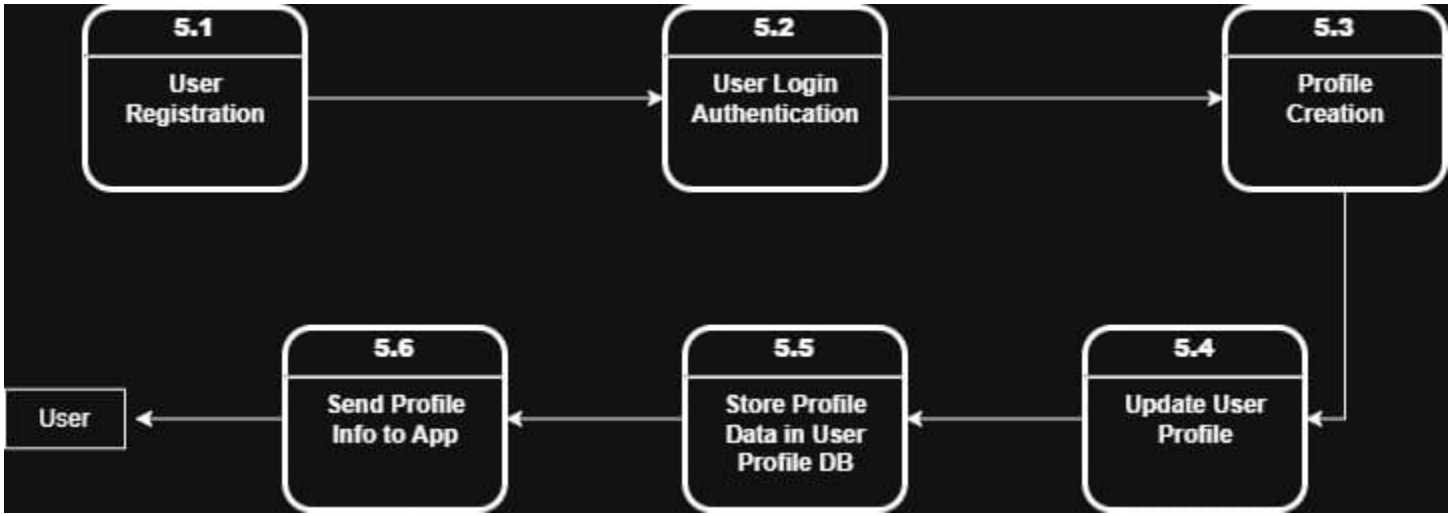
DFD diagram (Level 2, process 3):



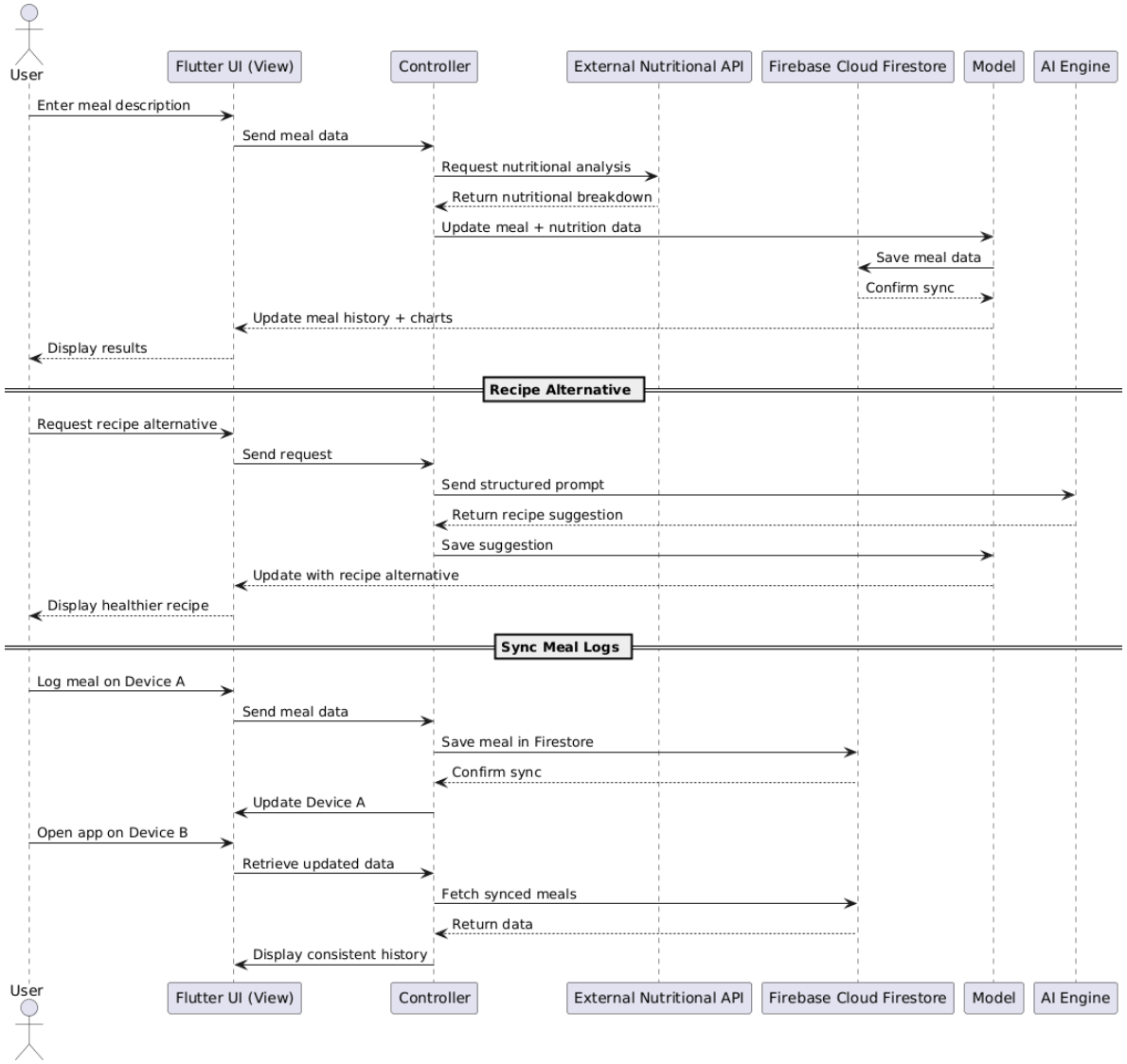
DFD diagram (Level 2, process 4):



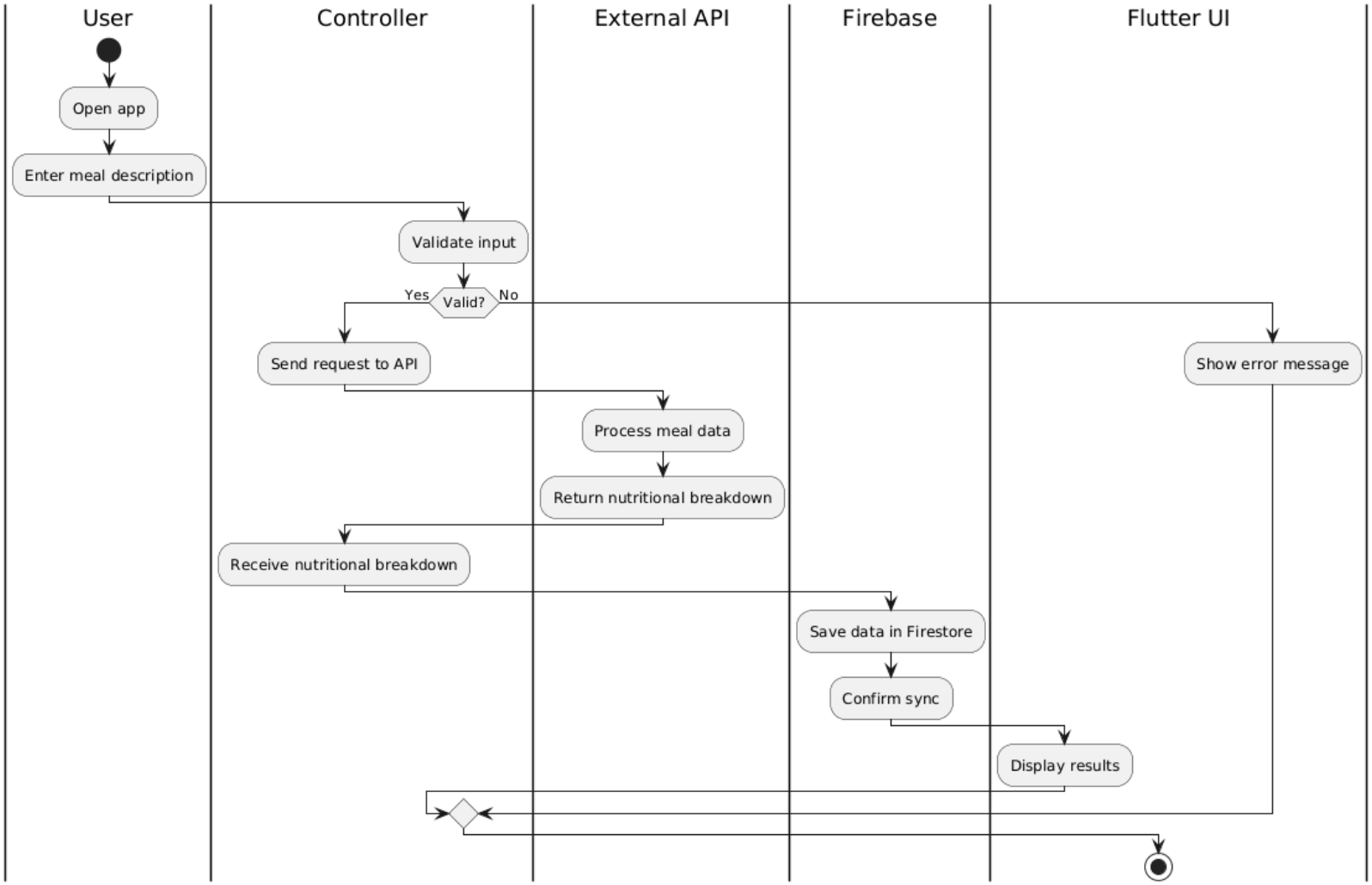
DFD diagram (Level 2, process 5):



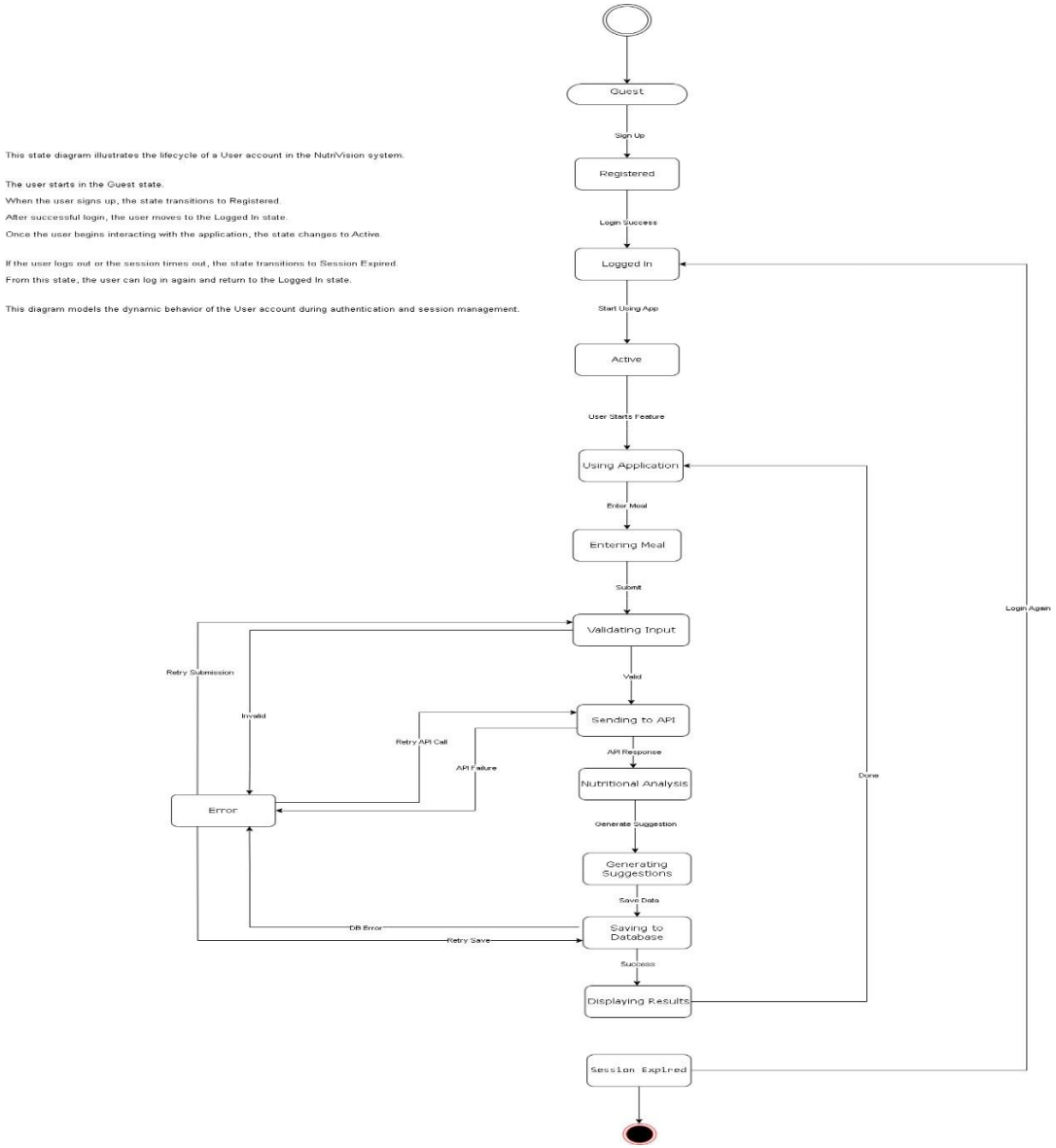
4.3.2. Sequence diagram:



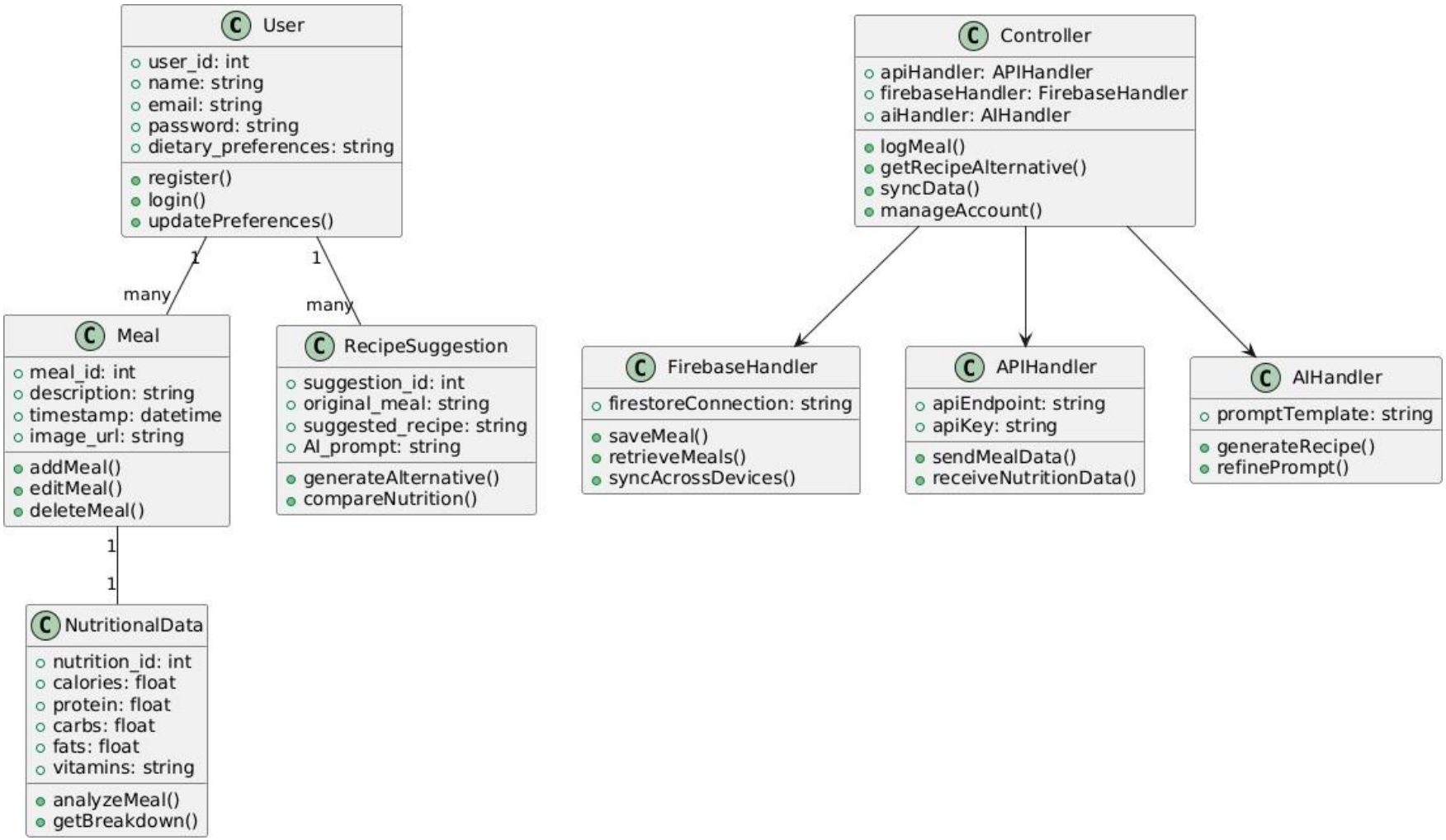
4.3.3. Activity diagram:



4.3.4. State transition diagram:

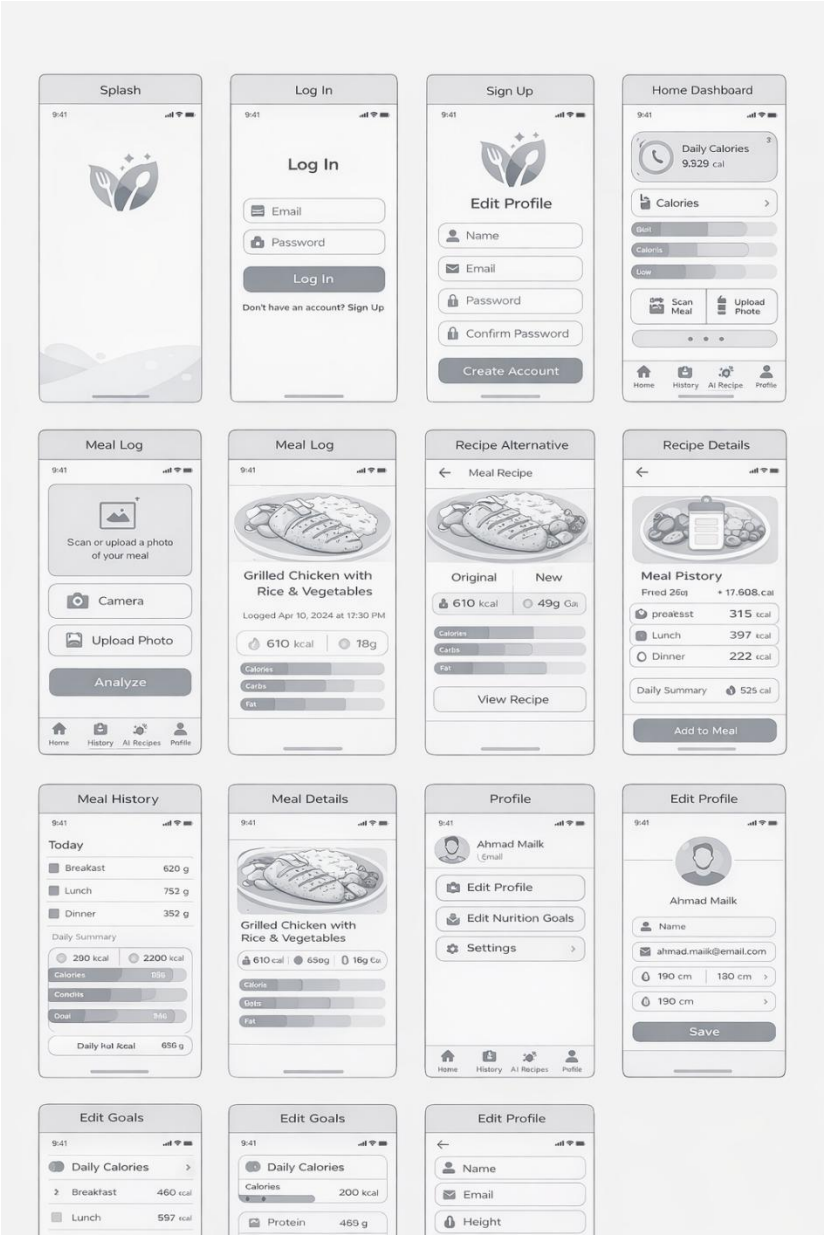


4.3.5. Class diagram:



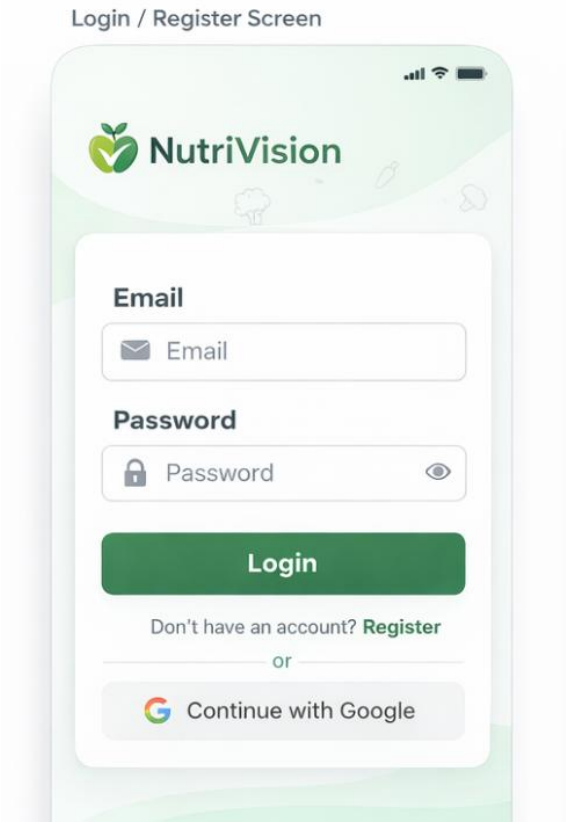
4.4. UI/UX Design & Prototyping:

4.4.1. Wireframes & Mockups:
4.4.1.1 Low-fidelity wireframes:

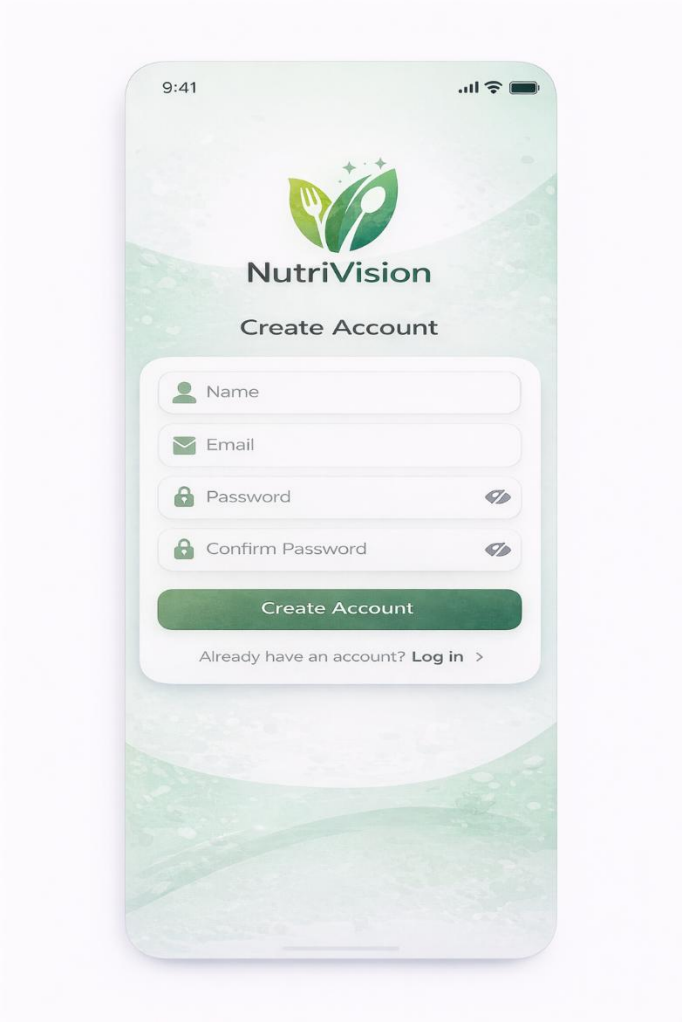


4.4.1.2.1 High-fidelity splash screen:

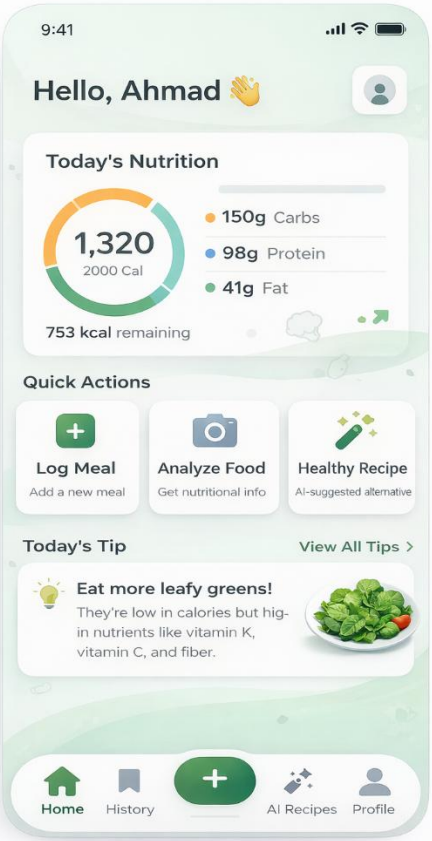
4.4.1.2.2 High-fidelity Sign in – Log in screen:



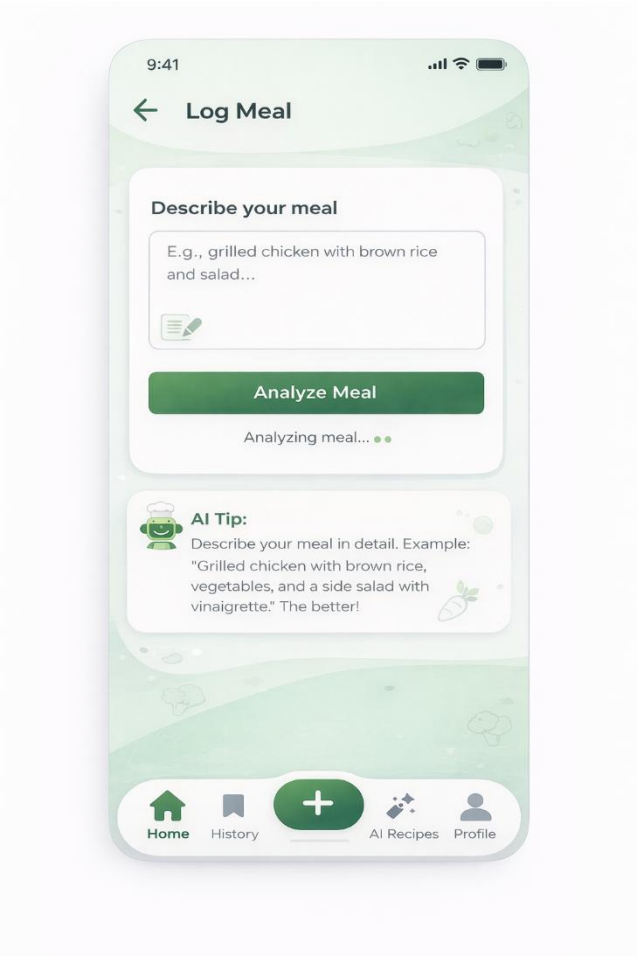
4.4.1.2.3 High-fidelity Sign-up screen:



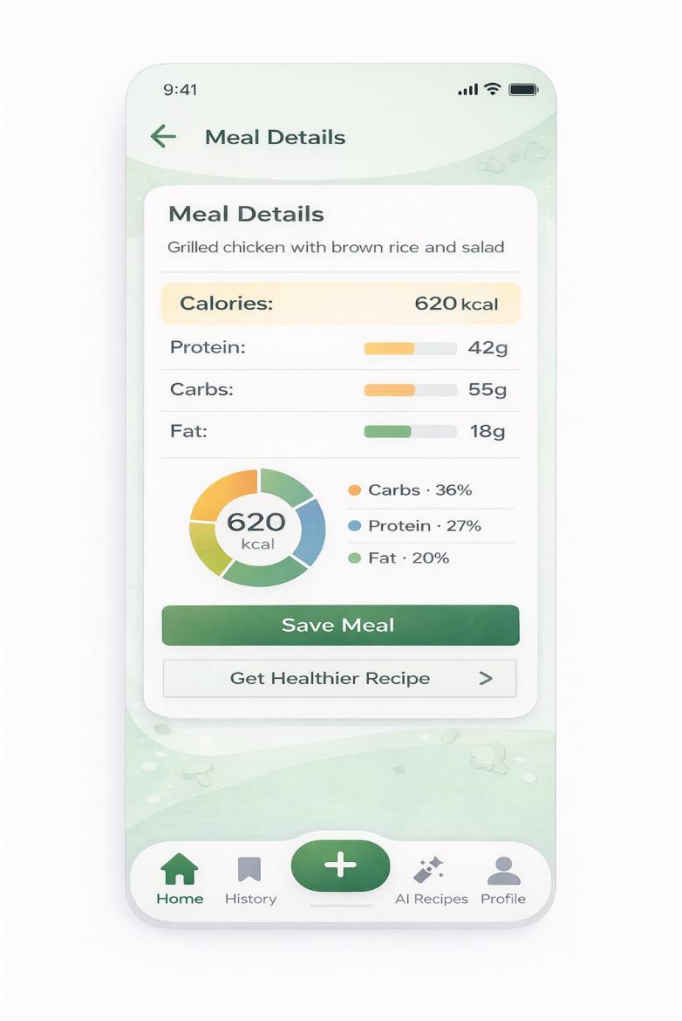
4.4.1.2.4 High-fidelity home dashboard screen:



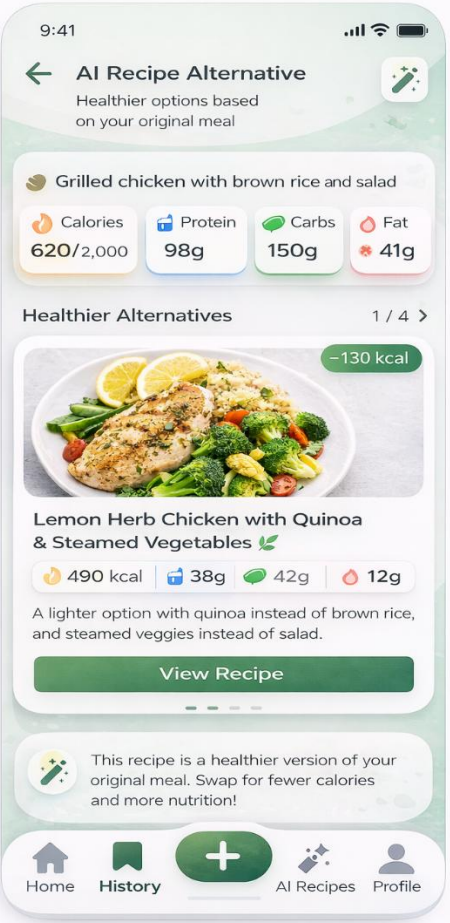
4.4.1.2.5 High-fidelity meal log screen:



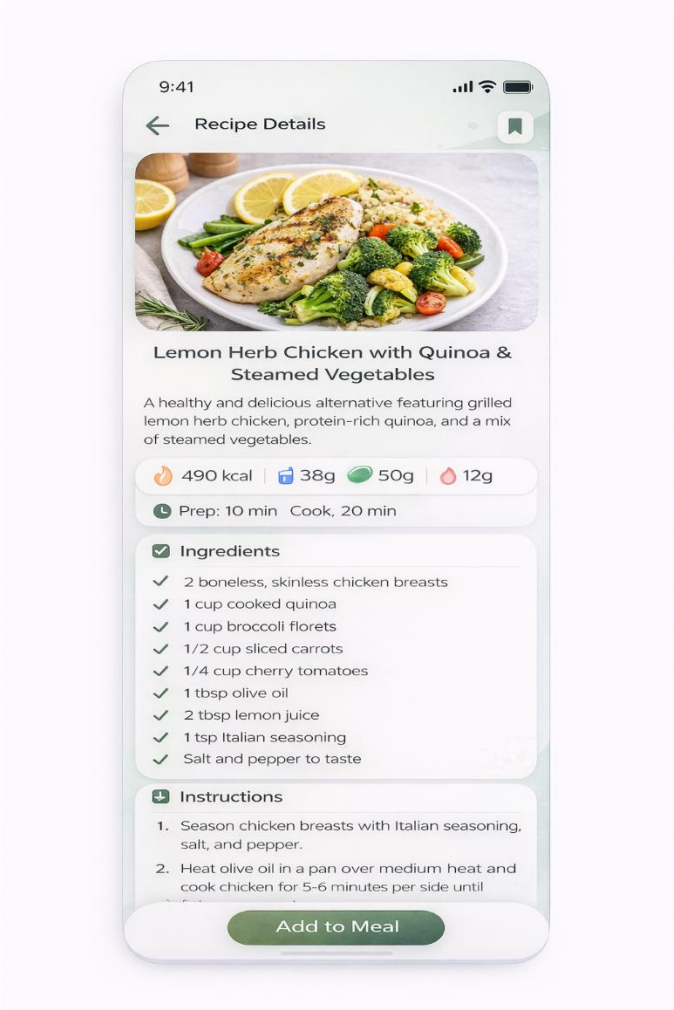
4.4.1.2.6 High-fidelity meal results screen:



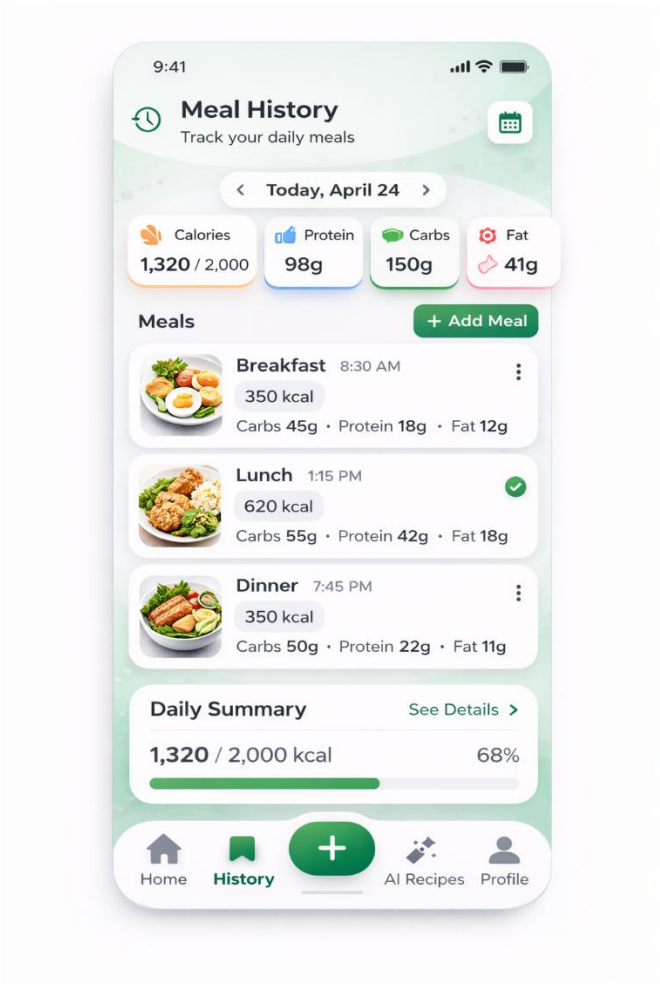
4.4.1.2.7 High-fidelity AI recipe screen:



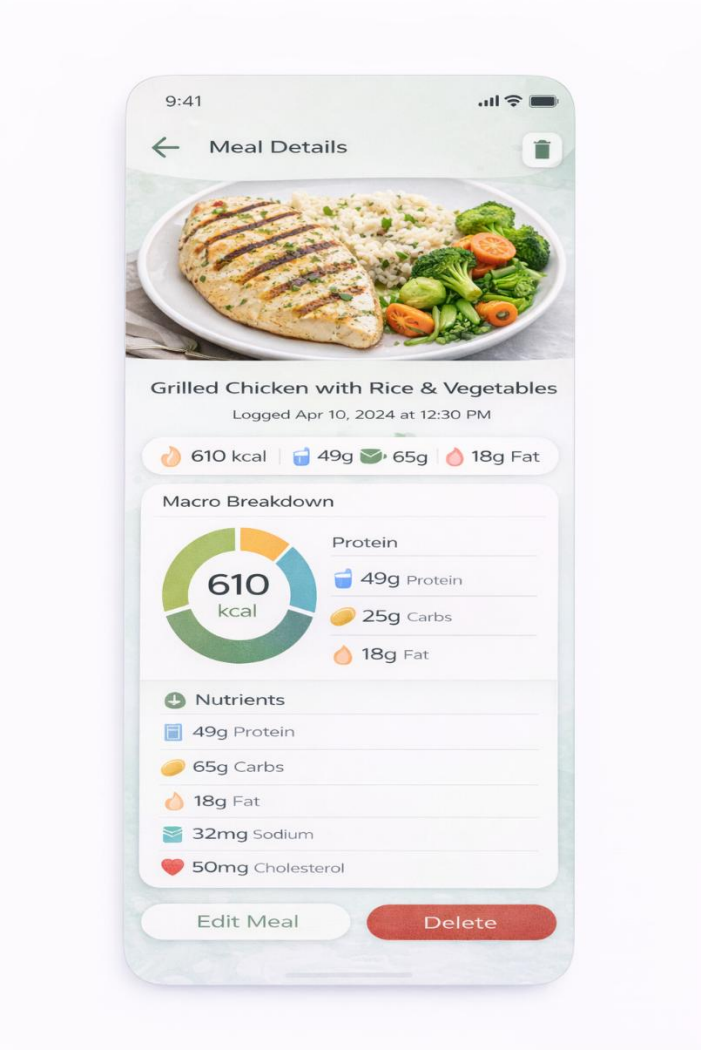
4.4.1.2.8 High-fidelity AI recipe details screen:



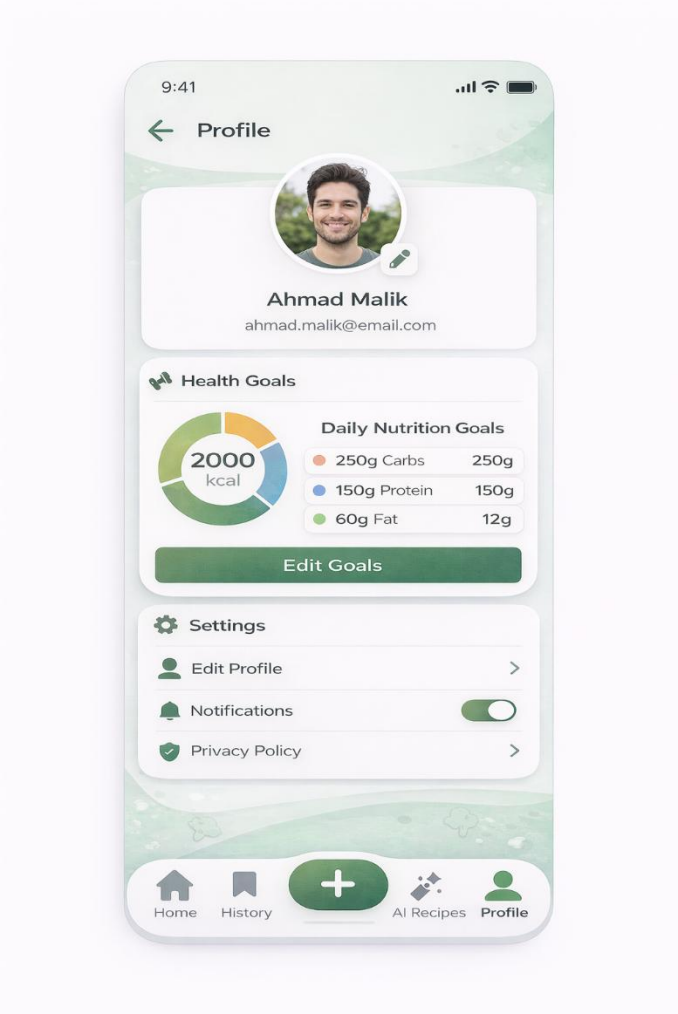
4.4.1.2.9 High-fidelity meal history details screen:



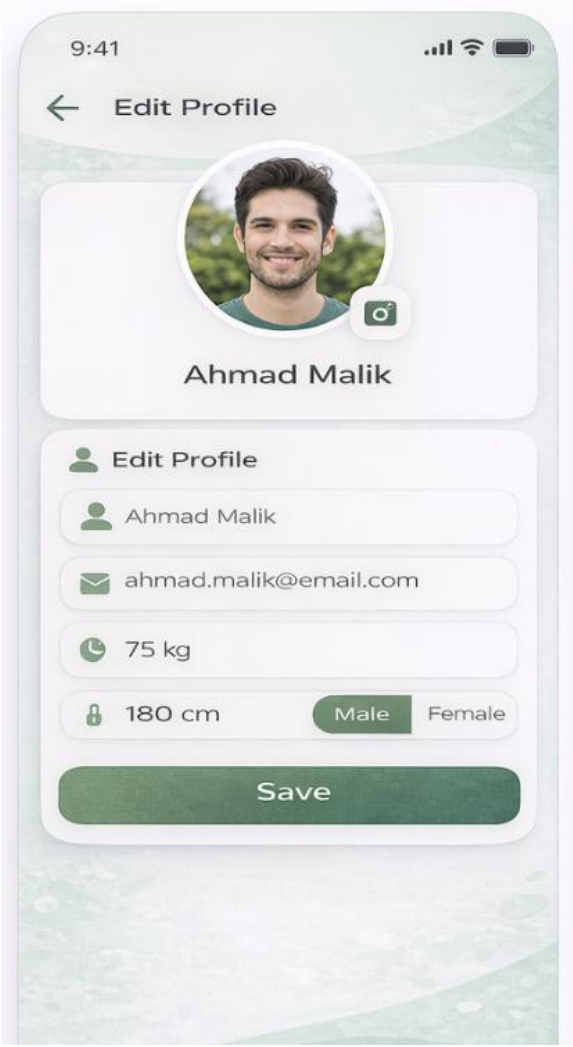
4.4.1.2.10 High-fidelity meal details screen:



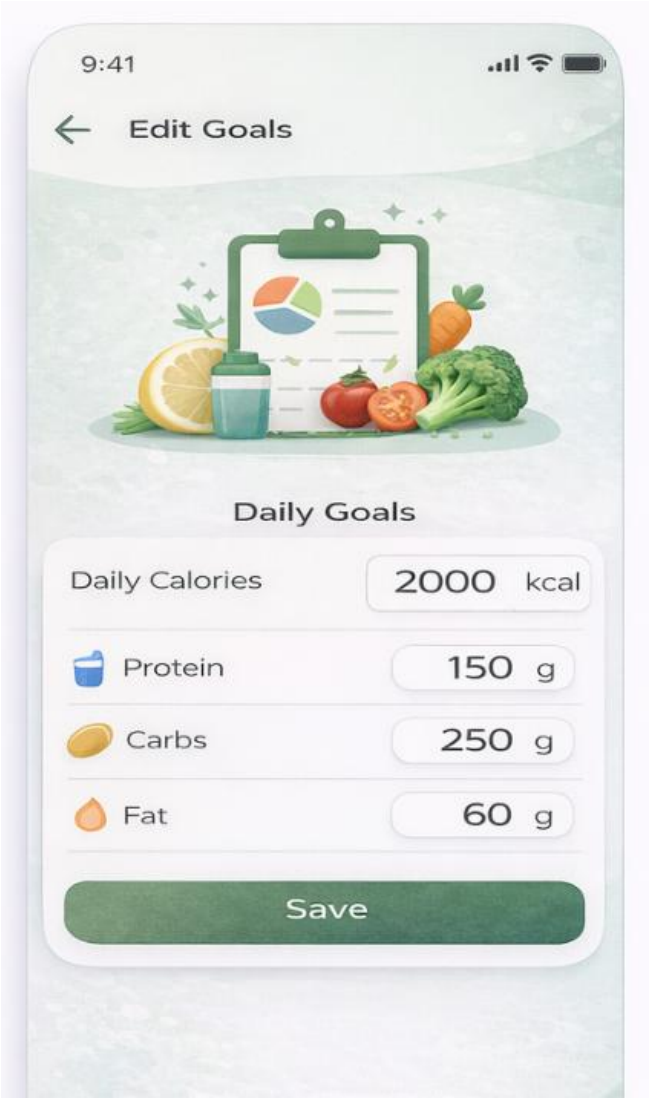
4.4.1.2.11 High-fidelity profile screen:



4.4.1.2.12 High-fidelity edit profile screen:



4.4.1.2.13 High-fidelity edit goal screen:



4.4.2. UI/UX Guidelines:

4.4.2.1. Introduction

The User Interface (UI) and User Experience (UX) design of the NutriVision mobile application focuses on creating an intuitive, visually appealing, and efficient interaction between users and the system. Since the application aims to help users track meals and understand nutritional information, the interface was designed to present complex data in a clear and user-friendly manner.

The design process follows modern mobile design standards, ensuring consistency, usability, and accessibility across all screens of the application.

The UI design also supports the integration of AI-generated nutritional insights and data visualization.

4.4.2.2 Design Objectives

The main objectives of the UI/UX design are:

- Provide a simple and intuitive interface for users.
- Enable users to log meals quickly and easily.
- Present nutritional data in a clear and understandable format.
- Maintain visual consistency across all screens.
- Ensure the application is accessible to a wide range of users.
- Support smooth navigation between different sections of the application.

These objectives guide the design decisions made during the UI development phase.

4.4.2.3 Design Principles

The NutriVision interface follows several key design principles to improve usability and visual quality.

3.1 Simplicity

The interface adopts a minimalist approach, displaying only essential information on each screen. This reduces cognitive load and allows users to focus on the most important tasks such as logging meals and viewing nutritional results.

3.2 Consistency

Consistency is maintained across all screens to create a predictable user experience. This includes consistent use of colors, typography, button styles, and navigation elements.

Consistent design helps users quickly understand how the application works and reduces the learning curve.

3.3 Visual Hierarchy

Visual hierarchy is used to guide the user's attention toward important information. Larger fonts, bold text, and highlighted colors are used to emphasize key elements such as calorie counts and nutritional breakdowns.

3.4 Accessibility

Accessibility considerations ensure that the application can be used comfortably by different users. These considerations include:

- readable font sizes
- clear contrast between text and background
- touch-friendly button sizes
- simple navigation patterns

3.5 Feedback and Responsiveness

The application provides visual feedback for user actions. For example, buttons highlight when pressed, loading indicators appear during AI processing, and confirmation messages are displayed when data is saved.
Providing feedback improves user confidence and interaction with the system.

4.4.2.4 Color Palette

The NutriVision application uses a carefully selected color palette that reflects themes related to health, nutrition, and well-being.

Primary Color

The primary color used in the application is green, which represents health, freshness, and natural food.
Example:
Primary Green: **#4CAF50**
This color is used for:

- primary action buttons
- highlights
- progress indicators
- key UI elements

Secondary Colors

Secondary colors are used to represent nutritional components and additional interface elements.
Examples include:

- Blue for protein indicators
- Orange or yellow for carbohydrates
- Red for fats

These colors help users visually distinguish between different nutritional categories.

Background Colors

The application primarily uses light backgrounds such as white and light green tones to create a clean and modern interface. Card-based layouts with subtle shadows are used to separate content sections.

4.4.2.5 Typography

Typography plays an important role in ensuring readability and clear communication of information.
The application uses modern, clean fonts that are easy to read on mobile devices.
Typography guidelines include:

- large and bold headings for titles
- medium-sized fonts for important information
- smaller fonts for supporting text
- consistent spacing between text elements

These practices create a clear visual hierarchy that helps users quickly understand the information displayed on each screen.

4.4.2.6 Iconography

Icons are used throughout the application to visually represent different functions and improve usability.
Examples include:

- home icon for the dashboard
- history icon for meal tracking
- food icons for meal logging
- profile icon for user settings

Using recognizable icons allows users to quickly understand the purpose of different features.

4.4.2.7 Navigation Design

Navigation is designed to be simple and intuitive. The application primarily uses a bottom navigation bar to allow users to easily move between the main sections of the application.
The main navigation sections include:

- Home Dashboard
- Meal History
- AI Recipe Alternatives
- User Profile

Additional navigation elements such as back buttons and action buttons help guide users through different workflows within the application.

4.4.2.8 Data Visualization

Because the application deals with nutritional data, visual representations are used to help users interpret information quickly.
Examples of data visualization include:

- nutrition charts
- macro nutrient breakdown diagrams
- progress bars showing daily calorie intake

These visual elements make complex data easier to understand compared to raw numerical values.

4.4.2.9 Mobile-First Design Approach

The NutriVision interface is designed using a mobile-first approach. This ensures that the layout and interaction patterns are optimized for smartphone devices.

Design considerations include:

- vertical scrolling layouts
- large touch-friendly buttons
- simple screen transitions
- optimized spacing for small screens

This approach improves usability and ensures a comfortable mobile experience.

4.4.2.10 Design Tools and Technologies

Several tools and technologies were used during the UI/UX design phase of the project.

Design Tools

The following tools were used to create the interface designs:

- AI-based UI generation tools for high-fidelity screen prototypes
- wireframe design tools for low-fidelity layouts
- digital mockup tools for visualizing the application interface

Development Framework

The application interface will be implemented using the **Flutter framework**, which allows the development of modern and responsive mobile applications using a single codebase.

Flutter provides powerful UI components and layout systems that support the design requirements of the NutriVision application.

4.4.2.11 Designed Screens

The following screens were designed during the UI phase of the project:

1. Splash Screen
2. Login Screen
3. Sign Up Screen
4. Home Dashboard
5. Meal Log Screen
6. Meal Result Screen
7. AI Recipe Alternative Screen
8. Recipe Details Screen
9. Meal History Screen
10. Meal Details Screen
11. Profile Screen
12. Edit Profile Screen
13. Edit Goals Screen

These screens represent the full user flow of the application and support all primary system functions.

4.5. System Deployment & Integration:

4.5.1. Technology stack:

1. Frontend

- Framework: Flutter
  - Cross-platform UI toolkit for Android, iOS, and web.
  - Provides responsive UI components (GridView, Stack, charts).
  - Ensures consistent design and smooth performance across devices.
- Language: Dart
  - Strongly typed, optimized for Flutter.
  - Supports reactive programming and state management.
- Libraries/Packages:
  - Provider/Bloc for state management.
  - Charts_flutter for nutritional visualization.

2. Backend

- Controller Layer (App Logic):
  - Handles API calls, AI prompts, and Firebase sync.
  - Built in Dart within Flutter, but logically separated for modularity.
- API Communication:
  - Dio package for HTTP requests to external APIs.
  - Provides structured requests and error handling.
- AI Integration:
  - Custom AI Engine (could be OpenAI API or similar ML service).
  - Processes structured prompts for recipe alternatives.
- Authentication & Sync:
  - Firebase Authentication provides secure login and user management.
  - Firebase Sync ensures real-time updates across devices.

3. Database

- Primary Database: Firebase Cloud Firestore
  - NoSQL, document-based.
  - Stores meals, nutritional data, recipe suggestions, and user preferences.
  - Real-time synchronization across devices.

- Schema Alignment:
    - User Collection → user_id, name, email, preferences.
    - Meal Collection → meal_id, description, timestamp
    - NutritionalData Collection → linked to meals.
    - RecipeSuggestion Collection → linked to users and meals.
- Indexes:
    - On user_id, meal_id, timestamp for fast queries.

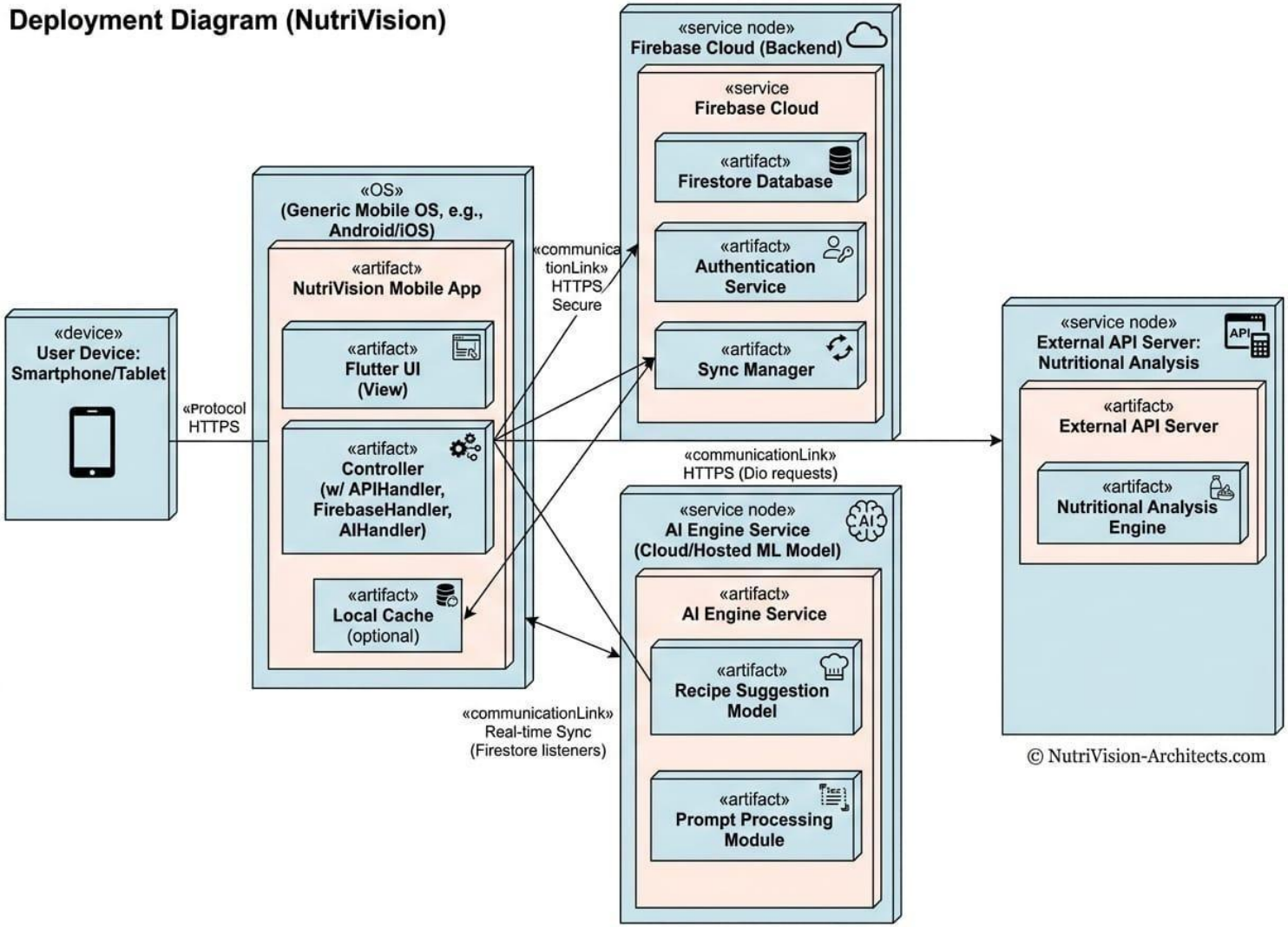
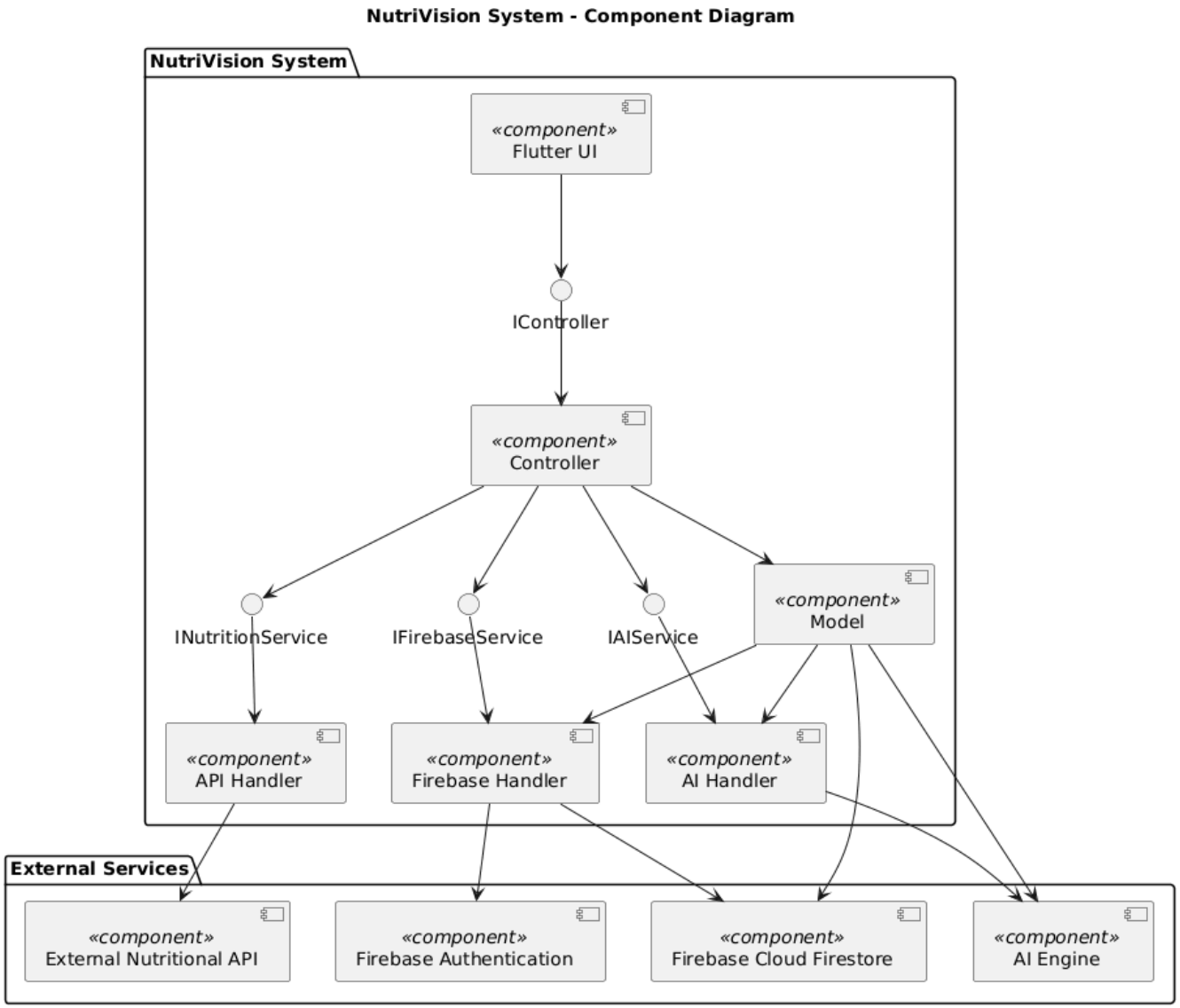
4. External Services

- Nutritional API:
    - Third-party service providing nutritional breakdowns.
    - Integrated via REST API calls.
- AI Engine:
    - Cloud-based ML service for recipe generation.
    - Receives structured prompts, returns alternatives.

5. Development & Deployment Tools

- Version Control: Git + GitHub/GitLab.
- CI/CD: GitHub Actions or Firebase Hosting pipelines.
- Testing: Flutter Test, Mockito for unit testing.
- Deployment:
    - Mobile → Google Play Store, Apple App Store /Web → Firebase Hosting.

4.5.2. Deployment diagram:



4.5.3. Component diagram: